

Measuring Execution

Guide for the DPI-LS Project

Digital Performance Index — Execution Dimension (E)

Architecture, Integration & Business Capabilities

Framework: Digital Performance Index (DPI-LS)

Dimension: Execution (E) — Weight: 20%

Version: 1.0

Status: Architectural Reference

DPI-LS Architecture Team

Contents

1	Executive Summary & Introduction	2
2	Execution Tracking Mind Map	2
3	The Execution Formula (How E is Calculated)	4
3.1	Calculation Example	4
4	Core Integration Architecture	6
5	Various Possible Ways for Measuring Execution (E)	6
5.1	Execution Integration Matrix	7
5.2	Execution Integration Decision Tree	8
5.3	Native Agent Framework Integrations	8
5.4	Direct API & Cloud Provider Integrations	9
5.5	OpenTelemetry & Third-Party Observability	9
5.6	AI Gateway & Proxy Integrations	11
5.7	Agent-Specific Observability	11
5.8	Enterprise APM Integrations	11
6	Business Scenarios: Real-World Execution Failures	13
7	Business Capabilities	14
8	Conclusion & Next Steps	15

1 Executive Summary & Introduction

NOTE

The **Digital Performance Index (DPI-LS)** framework quantifies AI agent performance across 7 distinct dimensions. Within this framework, the **Execution (E)** dimension tracks an agent's technical reliability and tool-calling success rate.

As AI Agents move from chat interfaces to autonomous workflows, they rely heavily on external tools, API calls, and code execution. When an agent attempts to execute a tool (e.g., querying a database, calling an API, writing to a file), it can either succeed or fail due to hallucinations, malformed JSON, or network errors.

This document provides the architectural guidelines for measuring the **Execution** dimension. It evaluates how we track agent function calls, how we integrate these telemetry points into the DPI-LS engine, and how measuring execution directly reflects on the operational resilience of the agent.

2 Execution Tracking Mind Map

The following mind map outlines the core requirements of measuring AI Agent Execution within the DPI-LS framework.



3 The Execution Formula (How E is Calculated)

The Execution dimension relies on a transparent mathematical formula calculated natively within the DPI-LS engine. It evaluates the agent's ability to successfully execute its planned actions.

IMPORTANT

Formula: $E = \frac{\text{successful_executions}}{\text{total_attempts}}$

- **total_attempts:** The total number of times an agent tries to execute a tool, function, or API call.
- **successful_executions:** The number of those attempts that return a successful 200 OK or valid operational state without crashing or failing.

3.1 Calculation Example

Suppose an AI Agent makes **14 total attempts** to use tools during a complex reasoning workflow:

Attempt Sequence	Status	Outcome	Recorded Success
Attempts 1-10	Success	Fetches correct records	10
Attempt 11	Failure	Hallucinates missing parameter	0
Attempt 12	Success	Self-corrects and retries	1
Attempt 13	Failure	External service times out	0
Attempt 14	Success	Succeeds on second try	1

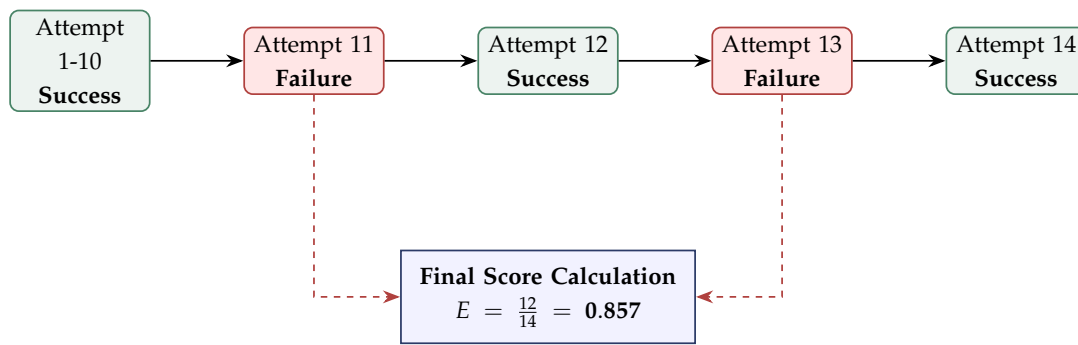
💡 TIP The Math:

$$\text{successful_executions} = 12$$

$$\text{total_attempts} = 14$$

$$E = \frac{12}{14}$$

$$\text{Final E-Score} = 0.857$$

**The Math:**

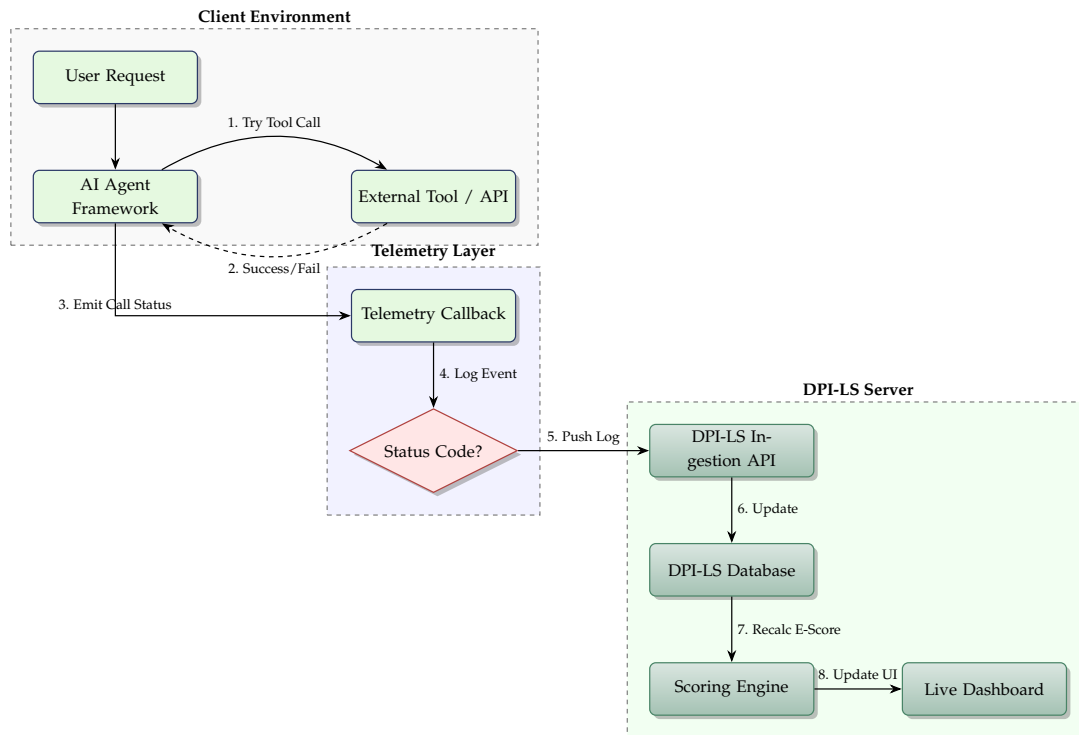
$$E = 12/14 = 0.857$$

Final E-Score = 0.857

An agent with an execution score of 1.0 flawlessly calls APIs and tools without needing to self-correct or retry. A lower score indicates the agent is struggling with syntax, hallucinations in function calling, or interacting with unstable external environments.

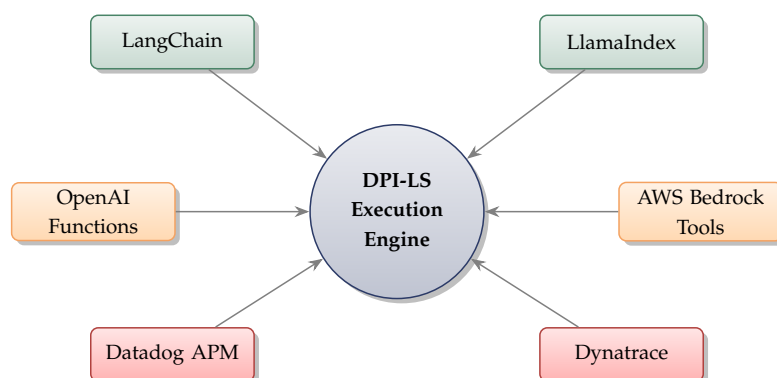
4 Core Integration Architecture

To reflect an agent's current Execution score on the DPI-LS dashboard, the agent framework must hook into the DPI-LS telemetry ingestion API to report tool attempts and their statuses.



5 Various Possible Ways for Measuring Execution (E)

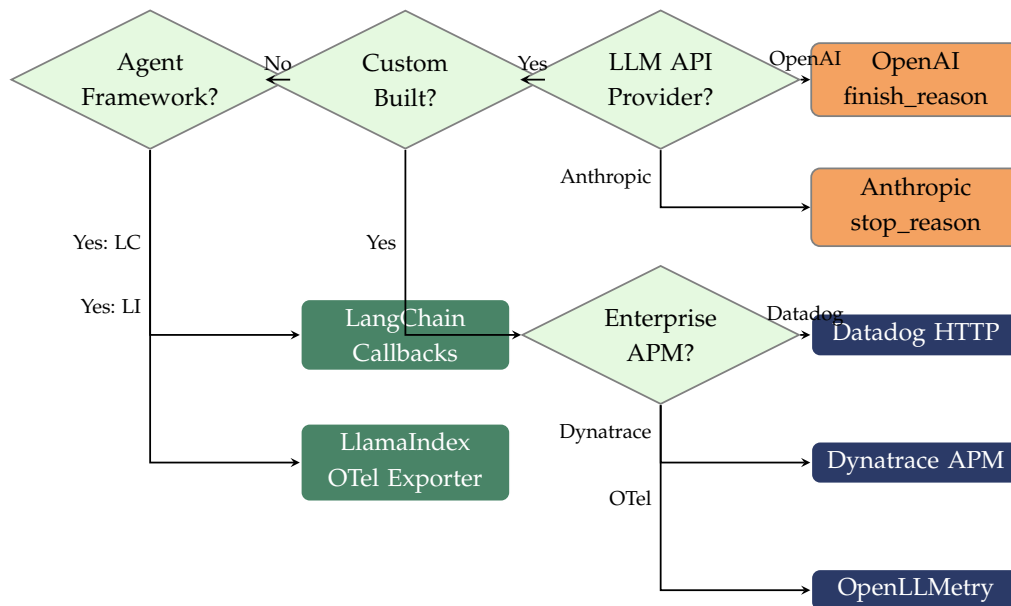
The DPI-LS framework is highly flexible, providing multiple avenues for capturing the E-score depending on the agent framework and infrastructure. The matrix below summarizes the supported methodologies:



5.1 Execution Integration Matrix

Execution Data	What It Measures	Source System	Actual Data Comes From
Agent graph tracing & debugging	Tool execution attempts and errors	LangSmith	LangSmith SDK pushes traces to webhook
Framework-native callback tracking	Tool execution attempts and errors in Python	LangChain	Custom BaseTracer injected into execution
Query pipeline telemetry	Query engine success rate	LlamaIndex	OTel Exporter to DPI-LS API
LLM-native function resolution	Successful JSON payload generation	OpenAI API	Parsing completion response finish_reason
Application performance monitoring	Low-level API network success	Datadog	HTTP 4xx/5xx logs correlated to agent traces
Standardized OTel LLM telemetry	GenAI Semantic Conventions spans	OpenLLMetry	Traceloop SDK exports
Full-stack trace analytics	Asynchronous agent execution graphs	Langfuse	Langfuse SDK exports
OpenInference AI observability	LLM tool span evaluation	Arize Phoenix	Phoenix OTLP exports
Enterprise span logging & evaluation	Tool execution trace logging	Braintrust	braintrust SDK logging
Chain-of-thought error tracking	Errors in multi-step workflows	Galileo	galileo-observe SDK
Open-source LLM tracing	Trace capture and debugging	Opik (by Comet)	Opik Python SDK / REST API
Multi-agent session tracking	Multi-agent execution loops	AgentOps	Native hooks into CrewAI / AutoGen
Gateway-level API routing logs	Intercepted HTTP headers at the edge	Helicone / Portkey	AI Proxy routing
Code-level function decoration	Wrapped Python functions	Custom App Code	@dpi_track_execution decorator

5.2 Execution Integration Decision Tree



5.3 Native Agent Framework Integrations

LangSmith

- **What it is:** The premier observability platform built by LangChain to track, debug, and monitor complex agentic workflows.
- **How can we integrate and use this for DPI-LS project:** LangSmith captures the complete execution flow of an agent, including every tool invocation, retrieval step, and reasoning logic. If a tool fails, LangSmith natively logs the exception, the exact inputs that caused it, and the latency.
- **How to wire it up:**
 1. Enable tracing in the environment via `LANGCHAIN_TRACING_V2=true`.
 2. Use the LangSmith SDK to automatically capture all agent loops.
 3. Configure LangSmith automation rules to POST failed tool executions directly to the DPI-LS webhook.

LangChain & LangGraph

- **What it is:** The most popular open-source frameworks for building complex agentic architectures and stateful workflows.
- **How can we integrate and use this for DPI-LS project:** By implementing a custom `BaseCallbackHandler`, DPI-LS can hook directly into the `on_tool_start`, `on_tool_end`, and `on_tool_error` events. This allows native, precise tracking of exactly when an agent attempts a tool and whether it succeeded or crashed natively in Python.

- **How to wire it up:**
 1. Create a `DPIExecutionCallbackHandler(BaseCallbackHandler)` class.
 2. Implement `on_tool_error` to dispatch a failure payload to the DPI-LS Webhook.
 3. Pass the callback handler into the Agent Executor initialization.
-

LlamaIndex

- **What it is:** A data framework specifically designed for connecting custom data sources to LLMs for RAG and agentic querying.
 - **How can we integrate and use this for DPI-LS project:** LlamaIndex natively supports OpenTelemetry (OTel). If a query pipeline or tool execution fails, the OTel spans immediately register the exception.
 - **How to wire it up:**
 1. Configure the `OpenTelemetrySpanHandler` in the LlamaIndex dispatcher.
 2. Route the OTel exporter to the DPI-LS ingestor endpoint.
 3. Filter spans by type `TOOL_CALL` and aggregate successes vs failures for the E-Score.
-

5.4 Direct API & Cloud Provider Integrations

OpenAI Function Calling API

- **What it is:** Native tool use capabilities built directly into OpenAI models like GPT-4o.
 - **How can we integrate and use this for DPI-LS project:** For lightweight agents not using heavy frameworks, DPI-LS can monitor the raw API responses. If an agent outputs `finish_reason: "tool_calls"` but fails to provide a valid JSON schema matching the registered functions, it counts as an execution failure.
-

5.5 OpenTelemetry & Third-Party Observability

OpenLLMetry (by Traceloop)

- **What it is:** An open-source observability framework built on OpenTelemetry, specifically designed to track LLM execution and agent tool calling using GenAI Semantic Conventions.
- **How can we integrate and use this for DPI-LS project:** OpenLLMetry automatically instruments agent workflows (like LangChain or LlamaIndex) and treats the agent's entire execution flow as a distributed trace. If an agent's tool call crashes, the exception is recorded as a child span.
- **How to wire it up:**

1. Install the `traceloop-sdk` and initialize it in the application entry point.
 2. Use `@workflow` and `@task` decorators on custom tool functions to ensure they are captured as individual spans.
 3. Export the telemetry data to the DPI-LS backend using a standard OTel exporter, allowing DPI-LS to penalize the E-score upon detecting tool-call error spans.
-

Langfuse

- **What it is:** An open-source, full-stack LLM engineering platform that focuses on comprehensive trace analytics, prompt management, and evaluation.
 - **How can we integrate and use this for DPI-LS project:** Langfuse excels at visualizing complex, multi-turn agent execution graphs. It natively integrates with frameworks like LangChain, tracking the exact latency, cost, and tool-call sequence of every execution.
 - **How to wire it up:**
 1. Add the Langfuse callback handler to your agent loop.
 2. Map Langfuse's concept of a "Trace" to the DPI-LS Session ID.
 3. Route Langfuse's webhooks or export its datasets into the DPI-LS ingestion API to calculate the E-score.
-

Arize Phoenix

- **What it is:** An open-source AI observability platform that leverages the OpenInference standard to trace LLM applications using OpenTelemetry.
 - **How can we integrate and use this for DPI-LS project:** Phoenix provides deep visibility into retrieval (RAG) and tool-calling performance. If an agent calls a tool and the context retrieved is irrelevant or the execution fails, Phoenix captures this as an OpenInference span.
 - **How to wire it up:**
 1. Instrument the agent application with the `openinference-instrumentation` library.
 2. Point the OTLP exporter to a Phoenix server (or directly to a compatible DPI-LS collector).
 3. Analyze the trace spans for `ToolCall` events marked with `StatusCode.ERROR`.
-

5.6 AI Gateway & Proxy Integrations

Helicone & Portkey

- **What it is:** Open-source AI gateways that act as a proxy layer between your application and the LLM providers.
 - **How can we integrate and use this for DPI-LS project:** Gateways provide zero-code observability. Because they intercept the raw HTTP requests, they can instantly capture malformed tool-calls, timeouts, and token costs without requiring you to install heavy tracing SDKs in your agent logic.
 - **How to wire it up:**
 1. Change your agent's LLM Base URL to point to the Helicone or Portkey proxy instance.
 2. The gateway captures the `finish_reason` and full tool-call payload.
 3. Route the gateway's webhooks directly to the DPI-LS Engine to record E-score failures.
-

5.7 Agent-Specific Observability

AgentOps

- **What it is:** An open-source observability platform purpose-built for multi-agent frameworks (like CrewAI, AutoGen, and PraisonAI) rather than generic LLM calls.
 - **How can we integrate and use this for DPI-LS project:** AgentOps captures the "session replay" and causal chain of why an agent selected a tool. It natively tracks "Agent Loops" and "Time Travel Debugging", making it perfect for complex execution failures.
 - **How to wire it up:**
 1. Initialize `agentops.init()` at the start of your multi-agent script.
 2. AgentOps automatically wraps framework tool executions.
 3. Use the AgentOps REST API to pull failed execution graphs and push them to the DPI-LS metrics database.
-

5.8 Enterprise APM Integrations

Datadog / Dynatrace

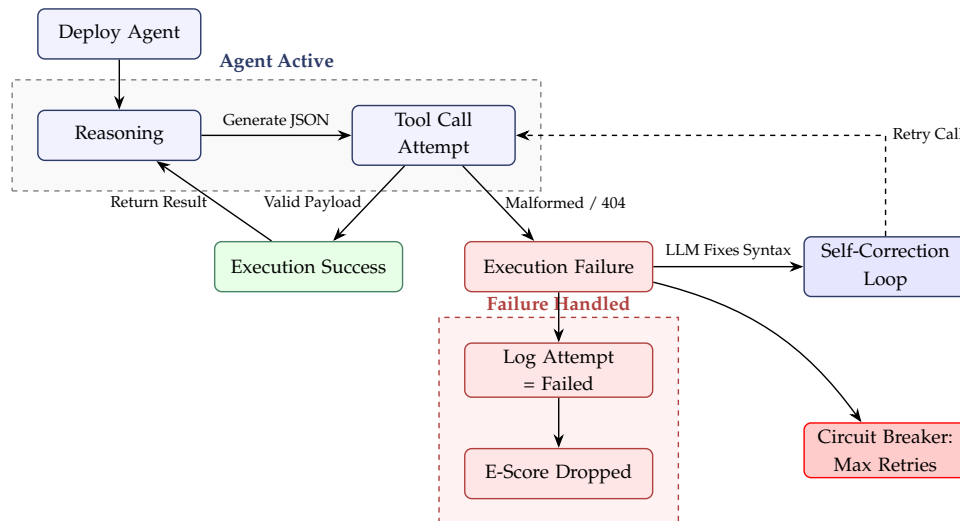
- **What it is:** Deep infrastructure monitoring platforms that track every network request, CPU spike, and HTTP status code in your cloud environment.
- **How can we integrate and use this for DPI-LS project:** Agents often fail because the *external API* they are trying to call is down (e.g. 504 Gateway Timeout, 500 Internal Server Error). APMs catch these network failures automatically.

- **How to wire it up:**
 1. Correlate APM traces with Agent Session IDs.
 2. When Datadog detects a 5xx error triggered by the agent's service, it fires a Webhook to DPI-LS.
 3. The DPI-LS engine records this as an Execution Failure and penalizes the E-Score.

6 Business Scenarios: Real-World Execution Failures

To illustrate the value of tracking tool execution natively within DPI-LS, consider the following enterprise scenarios where an agent attempts to utilize external functions.

When a tool fails, the agent must either self-correct or fail gracefully. The DPI-LS system tracks these transitions. The state diagram below visualizes this exact flow:



Scenario 1: The Hallucinated Syntax (Malformed JSON)

1. The Risk Event:

An autonomous data-entry agent attempts to write a record to a database, but the LLM hallucinates a parameter that doesn't exist in the tool's schema.

- **Agent Intent:** *"I will now insert the customer record using the `insert_db` tool."*
- **Agent Payload:** `{ "customer_id": 123, "email": "test@test.com", "imaginary_field": true }`

2. Automated Detection (How E is Detected):

The agent's framework intercepts the call and attempts to validate the payload against the Pydantic schema. It throws a `ValidationError` because `imaginary_field` does not exist. The tool call fails entirely.

3. Integration & Score Downgrade:

- **Endpoint Hit:** The tracing callback registers the failure and sends a payload to the DPI-LS backend.
- **Score Updated:** The DPI-LS scoring engine logs **1 failed attempt**. The agent's Execution (E) score drops.
- **Outcome:** The agent might receive the error message in its prompt and try again, but the initial failure remains permanently logged against its E-score.

Scenario 2: The Unstable External API (Network Timeout)

1. The Risk Event:

A financial forecasting agent successfully formats its JSON payload and calls an external stock-pricing API, but the external API is currently down.

2. Automated Detection (How E is Detected):

The HTTP request hangs for 10 seconds and throws a `TimeoutError`. Even though the agent did nothing wrong syntactically, the execution attempt failed.

3. Integration & Score Downgrade:

- **Endpoint Hit:** The Datadog APM integration catches the 504 Gateway Timeout and pushes a failure event to DPI-LS.
- **Score Updated:** The DPI-LS engine registers **1 failed attempt**, downgrading the E-score.

Scenario 3: The Seamless Self-Correction

1. The Risk Event:

An agent is asked to write a Python script and execute it in a sandbox. It writes the script, but forgets to import the `json` module.

2. Automated Detection (How E is Detected):

The Python sandbox tool returns the `NameError`. The DPI-LS ingestion API logs this as **1 failed attempt**.

3. Integration & Recovery:

- **Self-Correction:** The agent reads the `NameError`, realizes its mistake, adds `import json`, and tries again.
- **Score Updated:** The second attempt is successful. The DPI-LS engine logs **1 successful attempt**.
- **Outcome:** The final tally for this task is 1 success and 1 failure ($E = 1/2 = 0.50$). The agent completed the task, but the execution tracking proves it was inefficient and wasted compute cycles to get there.

7 Business Capabilities

By meticulously tracking the Execution dimension, organizations unlock several operational capabilities:

- **Self-Correction Visibility:** By looking at the ratio of attempts to successes, engineers can see how often an agent is forced to "retry" and "self-correct". An agent that succeeds on the 3rd try still gets the job done, but it is wasting compute and latency.

- **Model Selection Optimization:** Different LLMs have varying levels of proficiency in generating strict JSON or adhering to complex tool schemas. The E-score provides a direct, quantifiable metric to compare how well GPT-4o executes tools compared to Claude 3.5 Sonnet or open-source models.
- **Flaky Tool Identification:** If multiple agents have low E-scores when interacting with a specific internal API, the telemetry will highlight that the *tool* is likely flaky or poorly documented in its system prompt, rather than the agent being unintelligent.

8 Conclusion & Next Steps

IMPORTANT

The Execution (E) dimension strips away the subjective nature of LLM outputs and focuses strictly on **operational reliability**. Can the agent format a payload correctly? Can it handle network exceptions? Does it crash?

By integrating execution callbacks directly into your agent frameworks and piping them into the DPI-LS engine, you gain absolute clarity into the technical competence of your autonomous systems.

Next Steps:

1. **Define Webhooks:** Ensure the DPI-LS execution API (`/webhooks/execution`) is active.
2. **Configure Callbacks:** Install LangChain or LlamaIndex callback handlers in the agent framework to automatically push tool statistics.
3. **Monitor Dashboard:** Observe the live UI to pinpoint failing API calls or syntactical hallucinations in real-time.